# Master Product Requirements Document (PRD)

**Product Name:** EDITH Next-Gen AML Intelligence Platform (formerly iShield)
**Target Architecture:** Modular Microservices (.NET 10 LTS + React Micro-Frontends + Kafka)
**Target Market:** GCC Financial Institutions, Exchange Houses, and Commercial Banks
**Screening Scope:** English Optimized
**Performance SLA:** 2 to 5-second end-to-end multi-pass screening budget
**Database Footprint:** Dual-Database (PostgreSQL 17 + OpenSearch 2.x — Zero In-Memory Redis Dependency)
**Version:** 3.0

---

## 1. Executive Summary & Strategic Scope

The next-generation EDITH AML Intelligence Platform modernizes the legacy monolithic ASPX application into a cloud-native, modular microservices platform running on **.NET 10 LTS** and **React Micro-Frontends**. The platform is engineered to eliminate alert fatigue, accelerate compliance investigations, and automate regulatory reporting for financial institutions operating under FATF-aligned central bank mandates (such as CBUAE, SAMA, and Central Bank of Oman).

### Core Business Objectives

- **Plug-and-Play Modularity:** Decouple platform capabilities into isolated core modules and optional add-ons, enabling feature

activation, removal, and licensing tiers without modifying the underlying codebase.
- **True Asynchronous Microservices:** Ensure the UI remains completely non-blocking for all personas. Heavy background processes (bulk screening, reporting, delta syncs) execute asynchronously while users continue navigating the platform.
- **Seamless Core System API Integration:** Provide a robust, dual-mode API Gateway (Synchronous + Webhook Callbacks) for flawless integration with internal Core Banking Systems.
- **Streamlined Dual-Database Footprint:** Eliminate Redis and in-memory key-value stores. Rely exclusively on PostgreSQL for relational integrity, paired with OpenSearch for high-throughput phonetic/fuzzy entity resolution.
- **English-Optimized Screening Engine:** Deliver deterministic and fuzzy entity resolution over 15+ million records within a 2 to 5-second latency window.

---

# 2. Modular Architecture & Core System Integration

The system implements an Event-Driven Modular Architecture utilizing **CQRS (Command Query Responsibility Segregation)** to ensure heavy read operations (dashboards) never block write operations (screening new transactions).

## 2.1. Core Banking System (CBS) API Integration Strategy

The platform connects to the institution's Core Banking/Transaction system via a robust, two-way API Gateway engineered for financial fault-tolerance.

- **Dual-Mode API Execution:**
  - **Synchronous Pre-Clearance (2-5s SLA):** The Core system sends `POST /api/v1/screen/transaction`. EDITH executes the 5-pass OpenSearch engine and

returns `CLEARED` or `HELD` immediately.
    - **Asynchronous Webhook Callbacks:** If a transaction is `HELD`, the Core system releases its threads. When an MLRO finally reviews the case and clicks `[Release]` or `[Freeze]` days later, the EDITH Notification/Event Hub fires a cryptographically signed Webhook back to the Core system (`POST /core/webhooks/aml-decision`) to execute the final ledger action.
- **Idempotency Execution:** Every incoming request from the Core system requires an `Idempotency-Key` header. In the event of a network timeout, if the Core system retries the identical payload, EDITH returns the cached result without triggering duplicate alerts or case records.

### 2.2. Schema-per-Module Isolation & Autonomous Migrations (PostgreSQL)

Every module manages its own isolated PostgreSQL schema and dedicated Entity Framework Core `DbContext` (`core`, `screening`, `cases`, `rules`, `notifications`). Each module runs its own migrations dynamically upon activation.

### 2.3. Zero-Downtime Hot Swapping & Distributed Deployment

- **Backend (.NET 10):** The host scans the tenant plugin directory and instantiates plugins inside collectible `AssemblyLoadContext` instances, enabling runtime upgrades without taking down the screening engine.
- **Distributed Microservices Topology:** Modules deploy into dedicated, horizontally scaled Linux Docker containers orchestrated by Kubernetes, communicating asynchronously over Kafka/RabbitMQ.

---

## 3. Dual-Database Persistence Architecture

---

## (No Redis)

The data layer relies strictly on two optimized databases:

| Database / Store | Technology | Scope & Responsibility |
|---|---|---|
| **System of Record & Audit** | **PostgreSQL 17** | Master entity catalog, case workflows, customer KYC records, audit trails, user IAM, custom rule configurations, whitelist tables, and transactional history. |
| **Search & Screening Engine** | **OpenSearch 2.x** | Dedicated read-optimized search index handling phonetic, fuzzy, and multi-attribute entity resolution over 15M+ records without impacting transactional performance. |

---

# 4. English-Optimized Multi-Pass Screening Engine (2–5s SLA)

### 4.1. The 5-Pass Screening Lifecycle

1. **Pass 1: Normalization & Cleansing (100–200 ms):** ASCII folding, punctuation removal, case folding, common corporate suffix stripping.
2. **Pass 2: Broad Retrieval (300–600 ms):** OpenSearch query pulls candidate pool (top ~500 matches).
3. **Pass 3: Phonetic & String Distance Scoring (500–1,200 ms):** Evaluates candidate pool against Double Metaphone, Beider-Morse Phonetic Matching, Jaro-Winkler, and Damerau-Levenshtein algorithms.
4. **Pass 4: Multi-Entity Evaluation (500–1,000 ms):** Simultaneously

screens Sender, Beneficiary, Intermediary/Correspondent Bank, and Corporate Directors.
5. **Pass 5: Deterministic Scoring & Whitelist Bypass (100–300 ms):** Calculates composite score; suppresses hits if active in the PostgreSQL Whitelist table; generates alerts if score meets the configurable threshold.

---

# 5. Dynamic Multi-Channel Notification Engine (`MOD-NOTIF-DYN`) [Opt-In Add-on]

Active push notifications are decoupled into an **opt-in Add-on module**. When licensed, this engine routes alerts based on dynamic, admin-controlled rulesets.

## 5.1. Admin Dynamic Routing & Advanced Sub-Features

- **Shift-Aware Presence Routing:** Syncs with HR shift schedules. Bypasses "Offline" officers and routes P1 alerts directly to "On-Shift" personnel to prevent SLA lapses.
- **Two-Way Actionable Webhooks:** Enables interactive Adaptive Cards in MS Teams. Officers can click `[Acknowledge P1 Alert]` inside Teams, securely syncing the state change back to PostgreSQL.
- **Liquid Template Engine:** Replaces static alert text with customizable Liquid templates for dynamic variable injection in emails/webhooks.

---

# 6. Next-Gen UI/UX Analyst Workspace (Hybrid Visual & Tabular)

## 6.1. Non-Blocking Asynchronous UX (The 202 Accepted Pattern)

No user (Admin, AML Officer, or Branch User) should ever be blocked by a loading screen during heavy processing.

- **Global Background Job Manager:** When a user initiates a heavy task (e.g., Bulk Screening, large report generation, or delta rescreening), the microservice instantly responds with an `HTTP 202 Accepted` status.
- **Task Queuing:** The UI immediately frees up, allowing the user to navigate to other pages, review cases, or chat.
- **SignalR Progress Indicator:** A unified "Active Jobs" widget in the top navigation bar displays live progress. Upon completion, SignalR pushes an actionable toast notification.

## 6.2. Collision Prevention: Real-Time Case Locking (1 Person Per Case)

To enforce strict regulatory audit integrity, only one analyst can modify a case at a time:

- **Live PostgreSQL Advisory Locks:** If Analyst A opens a case, their avatar appears on that queue row globally. A 🔒 icon indicates it is locked.
- **Read-Only Observer Mode:** If Analyst B opens the locked case, action buttons (`Dispose`, `Freeze`) are disabled. A banner reads: *"🔒 This case is currently being investigated by [Analyst A]."*

## 6.3. Interactive Diagnostic Suite

- **Hold Rationale: Circuit Breaker Diagnostic:** A pipeline visual showing exactly which rule (e.g., Velocity Anomaly vs. Sanctions Match) tripped and caused the transaction to freeze.
- **Identity Comparison Engine:** Side-by-side diff viewer highlighting exact string matches in **Red**, phonetic variances in **Amber**, and complete mismatches in **Gray**.
- **Activity Time Machine:** Subway-style visual case lifecycle map

replacing raw timestamp tables.

---

# 7. Master Feature Matrix & Packaging

| Module Code | Module / Feature Name | Package Tier | Modernized Implementation |
|---|---|---|---|
| `MOD-CORE-IAM` | Multi-Tenant IAM & RBAC | **Core** | OpenID Connect, OAuth2, dynamic menu entitlement loading, ABAC. |
| `MOD-CORE-API` | Core System Gateway | **Core** | Dual-mode API (Sync pre-clearance + Async Webhook callbacks) with Idempotency. |
| `MOD-SCRN-BASE` | Watchlist & Sanctions Screening | **Core** | 2–5s multi-pass English engine, Double Metaphone, BMPM, Jaro-Winkler. |
| `MOD-SCRN-BULK` | Bulk File Screening Engine | **Core** | Asynchronous streaming CSV/Excel processor returning 202 Accepted with SignalR toasts. |
| `MOD-RULE-PROC` | AML Transaction Processor | **Core** | Zero-latency real-time API screening and distributed batch job execution. |
| `MOD-CASE-WORK` | Tiered Case Management & STR | **Core** | Integrated 3-pane canvas, 3-day EDD SLA, and 1-Person strict case locking. |
| `MOD-NOTIF-DYN` | Dynamic Notification Engine | **Add-on** | Admin-controlled routing with toggleable Shift-Presence, Teams callbacks, and Diagnostics. |

| | | | |
|---|---|---|---|
| `MOD-GOAML-GCC` | Automated `goAML` Exporter | **Add-on** | Automated UNODC goAML XML payload builder meeting specific FIU schema requirements. |
| `MOD-AI-NARR` | GenAI STR Narrative Copilot | **Add-on** | Automated regulatory STR narrative drafting powered by LLM pipeline. |
| `MOD-AI-GRAPH` | Entity Relationship Visualizer | **Add-on** | Interactive Cytoscape.js canvas mapping UBOs, smurfing, and shared links. |

# 8. Architectural Considerations and Suggestions

## 8.1. Infrastructure and Deployment

- **Containerization:** All microservices should be containerized using Docker, allowing for consistent deployment environments and easy scaling.
- **Orchestration:** Use Kubernetes (K8s) for managing the containerized applications. It will handle load balancing, auto-scaling, and self-healing.
- **Service Mesh:** Consider implementing a service mesh (e.g., Istio or Linkerd) for advanced traffic management, security (mTLS between services), and observability.
- **CI/CD Pipeline:** Establish a robust CI/CD pipeline (e.g., GitHub Actions, GitLab CI, or Azure DevOps) for automated testing and deployment of microservices.

## 8.2. Data Management and Security

- **Data Encryption:** Ensure data at rest (in PostgreSQL and

OpenSearch) and data in transit (TLS 1.3) are encrypted.
- **Secrets Management:** Use a dedicated secrets management tool (e.g., HashiCorp Vault or Azure Key Vault) to store sensitive information like database credentials and API keys.
- **Database Partitioning:** Implement table partitioning in PostgreSQL for high-volume tables (e.g., `transactions`, `audit_ledgers`) to maintain query performance over time.
- **Backup and Disaster Recovery:** Establish automated, regular backups of the PostgreSQL database and OpenSearch clusters, with a clear disaster recovery plan.

## 8.3. Observability and Monitoring

- **Centralized Logging:** Implement a centralized logging solution (e.g., ELK stack or Datadog) to aggregate logs from all microservices for easier troubleshooting.
- **Distributed Tracing:** Use distributed tracing (e.g., Jaeger or OpenTelemetry) to track requests across the microservices architecture and identify performance bottlenecks.
- **Metrics and Dashboards:** Utilize Prometheus and Grafana for collecting system metrics and creating real-time operational dashboards.

## 8.4. Performance and Scalability

- **Caching Strategy:** While the core architecture avoids Redis for transactional data, consider using it strictly as a distributed cache for read-heavy, non-transactional data (e.g., UI configurations or reference data) if performance bottlenecks arise.
- **Asynchronous Messaging:** Ensure the Kafka message bus is properly tuned for high throughput and low latency. Use consumer groups for parallel processing of events.
- **OpenSearch Optimization:** Carefully design the OpenSearch mapping and indexing strategy to balance search accuracy (fuzzy matching) with response time, adhering to the 2-5s SLA